

Front matter (booklet v1, 2026-10-09)

Title. Grid vs. Graph Representations for Self-Play Learning in Hive: a Pre-Registered Comparison under Limited Compute

(Descriptive, no presumed result, per plan ch. 20.)

Author. Mohamed Bechir Kefi

Status. Independent research report — not peer-reviewed, not a publication of any institution. Version 1.0-draft, 2026-10-09.

Code and artifacts. Repository `hive-graph-selfplay` (release tag and access to be fixed at diffusion time, G-PUBLIC); results manifest under `results/`; every figure and table regenerates from scripts over raw per-game records.

AI assistance declaration. This study was executed under a human-as-PI methodology in which an AI research assistant (Claude, Anthropic) implemented code, ran campaigns, and drafted text under a gate system reserving all scientific decisions — freezes, budgets, spending, publication — to the human author, who owns every claim. The full division of labour is documented in the repository (`docs/methodology.md`) and summarised in the annexes.

Abstract (164 words). Hive is a boardless hexagonal strategy game whose moves are (piece, destination) pairs over an ever-changing set of cells — a natural candidate, in principle, for graph neural encodings over the convolutional grid encodings standard in AlphaZero-style systems. We test that intuition under a pre-registered protocol frozen before any comparison run: one grid CNN and one capacity-matched (+2.1%) relational message-passing network share a rules-validated engine, one action decoder, identical training settings, a frozen three-opponent population and 250 frozen openings, evaluated under two budget readings (equal training examples; equal wall-clock at a pre-registered cutoff) with five independent seeds per arm and truncation reported as its own outcome. The hypothesis is rejected: the graph arm scores lower against two of three opponents under both readings, at twice the wall-clock cost, with its deficit concentrated in converting won positions. Ablations show the graph arm's typed edge relations are load-bearing for optimization stability, while its global pooling is dispensable. Negative, pre-registered, and fully reproducible from released records.

Keywords. Hive; AlphaZero; graph neural networks; representation learning; pre-registration; negative result

Table of contents. 1 Introduction · 2 Formalisation · 3 Related work · 4 Method (engine validation; baselines; pipeline; representations) · 5 Protocol · 6 Results (comparison; ablations) · 7 Discussion and threats · 8 Conclusion · Bibliography · Webographie · Annexes (corpus, conventions, architectures, configs, seeds, reproduction commands)

Chapter 1 — Introduction (booklet draft, 2026-10-09)

Written last-but-abstract per the plan's order, from final results.

The question. Hive is a boardless hexagonal strategy game: pieces define the playing surface, stacks form and dissolve, and every move is a (piece, destination) pair over a set of cells that changes each turn. Grid-based encodings — the convolutional lingua franca of AlphaZero-style systems — must bolt a frame onto this frameless game. A graph encoding needs no frame at all. It is natural to expect the graph to win. Does it, at a compute budget a single machine can afford?

Why it is not obvious. The intuition cuts both ways. Graph networks match the game's native structure and carry no anchoring artifacts; but Hive's outcomes hinge on short-range surround tactics — the regime in which convolutional locality is strongest — and prior evidence splits: graph arms won in Hex under DQN (Keller et al. 2023) and in chess under self-play (Rigaux & Kashima 2024, from single runs), while the only AlphaZero-on-Hive study (de Goede et al. 2022) never tried a graph at all. The hypothesis was stated falsifiably and frozen before any comparison run, together with everything that could bend the answer: opponents, openings, budgets, evaluation settings, and the rejection rule itself.

What we did. We built both encodings behind one shared action decoder on one rules-validated engine, matched capacity to +2.1%, and trained five independent seeds per arm under identical self-play settings, reading the comparison two ways — equal training examples and equal wall-clock — against a frozen three-opponent population on 250 frozen openings, with truncation as a first-class outcome throughout.

What we found. The hypothesis is rejected. The grid arm scores higher against two of the three opponents under both readings (largest interval upper bound for a graph advantage: +0.028), at half the wall-clock cost; the graph arm's deficit concentrates in local tactical conversion — it wins material against the random opponent and then fails to close, truncating 20–57% of those games where the grid arm truncates almost none. Ablations localise the graph arm's machinery: its direction-typed edge relations are load-bearing for optimization itself (removing them diverges training in every seed), while its global pooling is dispensable.

Contributions. (1) The first controlled, budget-matched, multi-seed grid-vs-graph comparison for Hive, both arms under the same AlphaZero-style pipeline, with a pre-registered negative answer — Chapter 6 and `results/comparison/`. (2) A reproducible comparison harness for frameless, stacking games: a validated rules engine with cross-language golden-pinned encoders, a shared variable-action decoder, truncation-aware evaluation machinery, and frozen-artifact discipline — Chapters 4–5 and the released code. (3) A component attribution for the graph arm (edge typing = trainability; pooling = null) with an honestly labeled supplement — Chapter 6 (H-T3) and `results/ablations/`.

Each contribution is verifiable from the repository: every number in this booklet traces to a journal entry and regenerates from scripts over raw per-game records.

Chapter 2 — Formalisation (booklet draft, 2026-09-26)

Sources: frozen protocol v1.0 (D-020); docs/action-decoder.md (D-015); docs/representations/; engine architecture notes. No numbers here beyond registered ones.

2.1 The game and the studied variant

Hive is a two-player, perfect-information, zero-sum game played on an unbounded hexagonal tiling: there is no board — the pieces in play define the playing surface. We study the **base game only** (queen, spiders, beetles, grasshoppers, ants; 11 pieces per side), decision D-007. The engine implements the

Mosquito/Ladybug/Pillbug expansions and the tournament opening restriction (no queen on a player's first turn); the kernel is exercised by tests across all eight game types, but no study result involves an expansion.

2.2 State, actions, transition, result

A state s comprises: the placed pieces with their cells and stack levels (beetles may climb, producing stacks), the reserves, the side to move, the ply count, and the stun marker `last_moved` (relevant only under the Pillbug expansion; carried for kernel uniformity — it is part of the hashed position). The engine exposes s through the Universal Hive Protocol (UHP); its rules correctness is established independently of any learning component (Chapter 4).

An **action** is (piece, destination cell), which identifies any Hive move uniquely — walk-vs-throw collisions produce identical successor states — plus a distinguished *pass*, legal exactly when no move exists. The legal-action set $A(s)$ is produced by the engine's generator; both learned architectures receive it and normalise their policies over exactly $A(s)$ (the shared decoder, Chapter 4). Transitions are the engine's `play`; the game ends when a queen is fully surrounded (win for the opponent; simultaneous surround = draw), with additional draw by threefold repetition.

Official result vs experimental truncation. Self-play and evaluation games are stopped at a 300-ply cap. A capped game is **not** a draw: it is a fourth outcome class, *truncated*, carried through the data format (records), the training loss (truncated games are excluded from the value target), the match runner, all tables (separate rate column) and the analysis. The reward for learning is the terminal result from the side-to-move perspective: win +1, loss -1, draw 0; truncation contributes no value target.

2.3 Search and learning notation

Both arms use the same PUCT Monte-Carlo tree search: at a node, an evaluator returns priors over $A(s)$ and a scalar value $v \in [-1, 1]$ (side-to-move perspective); selection maximises $Q + c \cdot \text{prior} \cdot \sqrt{N}/(1+n)$ with $c = 1.4$; terminal values back up exactly. Self-play uses playout-cap randomization (a fraction 0.25 of decisions get 128 simulations and are recorded; the rest get 32 and are not), Dirichlet root noise ($\epsilon = 0.25$), temperature sampling for the first 12 plies, and resignation below -0.92 with a 10% no-resign audit fraction. **Evaluation applies none of the exploration machinery** ($\epsilon = 0$, deterministic argmax; enforced by test and pinned by config, D-019) and runs 400 simulations per decision.

Training minimises a policy cross-entropy against the recorded MCTS visit distributions, normalised over the legal set for both arms, plus a weighted 3-class value loss (weight 0.6) over win/draw/loss with truncated samples excluded. Budgets are accounted in four denominations per run — wall-clock, hardware, training states, simulations — and the comparison is read under two equalisations (same-examples, same-wall-clock), Chapter 5.

2.4 Game view vs encoding view

The same state s is encoded two ways (figure fig3-encodings): the *grid view* embeds the position in a fixed 32×32 frame (BFS-unwrapped from the engine's toroidal board, bounding-box-centred) with 77 feature planes; the *graph view* is coordinate-free — nodes are the occupied cells plus every empty cell adjacent to the hive (exactly the decoder's destination universe), edges carry the six hex directions as types, stacks appear level-by-level as node features, and reserves/side/ply enter as a global vector. Chapter 4 specifies both; nothing else in the system differs between the arms.

Chapter 3 — Related work and positioning (booklet draft, 2026-09-26)

Sources: `docs/reading/` (verified notes; one row per work in `docs/reading/matrix.md`); decision D-009 (scoped claim). Only noted, actually-consulted sources are cited; full metadata lives in the `bibliography/webographie`.

3.1 Self-play reinforcement learning at scale — and small scale

AlphaZero (Silver et al. 2017/2018) fixed the algorithmic template this study inherits: a policy-value network guiding PUCT-MCTS self-play. Its budget reporting, however, is hardware-denominated and single-run — precisely what a small-compute, multi-seed study must improve on. KataGo (Wu 2020) showed the pipeline’s cost is highly compressible and set the budget-reporting standard (GPU-days, games, samples) our accounting follows; its representation-agnostic economies (playout-cap randomization) are applied identically to both arms here. Jones (2021) demonstrated that deliberately small AlphaZero-style experiments on Hex yield lawful, extrapolatable signal — the closest methodological licence for our setting — and supplied the compute-frontier reporting style, while never varying the representation. Agarwal et al. (2021) provide the statistical frame: few-run regimes demand seed-level resampling and interval reporting, which our protocol adopts with the seed as the unit.

3.2 Graph representations and variable action spaces

GraphSAGE (Hamilton et al. 2017) grounds the inductive message-passing family our graph arm belongs to; vanilla GraphSAGE lacks edge semantics, which our direction-typed relations add. Pointer Networks (Vinyals et al. 2015) justify scoring a variable candidate set — our shared decoder scores exactly the legal (piece, destination) pairs, with empty destination cells as first-class scorable objects. MDP-homomorphic networks (van der Pol et al. 2020) motivate the *time-permitting* symmetry question only; per the protocol, no invariance is assumed of a GNN, and none was claimed.

3.3 Hive and grid-vs-graph comparisons

Scholarly Hive AI is thin. Kampert et al. (2021) built heuristic minimax/MCTS agents (BeeKeeper) and documented Hive’s ~60 branching factor and the weakness of naive evaluation signals. AZ-Hive (de Goede et al. 2022) is the closest prior work: AlphaZero on Hive across five board encodings — all dense grid/CNN, no graph arm — with engines that remained below plain search; its finding that encoding choice strongly affects learning is direct motivation for RQ-H1. Polygames (Cazenave et al. 2020) achieves boardsize invariance within the grid paradigm and does not support Hive. The direct grid-vs-graph precedents are Keller et al. (2023) — parameter-matched CNN-vs-GNN on Hex, but under RainbowDQN (the CNN arm never trained under MCTS self-play) and with a Hex-specific Shannon-game graph — and Rigaux & Kashima (2024, NeurIPS), an edge-featured GAT for chess reporting GNN gains in an AlphaZero-style loop — from a single training run per model, without seed replication. Ben-Assayag & El-Yaniv (2021) trained a GNN-AlphaZero on Othello for size scaling, not for a budget-matched representation comparison. An unpublished hobby project (hiveGo; webographie) trains a small GNN on Hive with an AlphaZero loop, with no controlled comparison.

3.4 Positioning

No prior work runs a controlled, budget-matched grid-vs-graph comparison for Hive with both arms under the same self-play pipeline; that is this study’s scoped contribution (D-009) — deliberately *not*

“the first grid-vs-graph comparison in a board game”, which Keller et al. and Rigaux & Kashima preclude. The scope closes as follows: one variant (base Hive), one grid net and one simple relational GNN at matched capacity, one machine, frozen opponents/openings/protocol, two budget readings, five seeds per arm. Within that perimeter the comparison is answered (Chapter 6); outside it, nothing is claimed. The contrast with Rigaux & Kashima’s positive chess result and Keller et al.’s long-range-vs-local asymmetry is taken up in the discussion.

Method — Engine correctness (report section draft)

Drafted 2026-09-09 (invariant 14: written as the phase runs). English working master; French copy at the report milestone (D-006). Every number traces to a journal entry (invariant 4); claims are registered in `claims.md`.

Why validation precedes everything

A self-play study trains on positions the engine itself generates and scores; a rules defect therefore does not merely add noise — it teaches both arms a systematically wrong game and silently invalidates the comparison (“false learning”, plan ch. 4). The stop rule is absolute: an engine found incorrect suspends all training-side work (invariant 11). We therefore validate along four independent lines before any baseline or training work, and report each line with its exact scope.

Four lines of evidence

1. Perfth against published tables. Node counts for all 8 game types match Mzinga’s published perfth tables: depth ≤ 5 in the standard test suite, depth 6 in a dedicated run (8.3 s, all types), depth 7 in the nightly script. Perfth is exhaustive within its depth: any movegen discrepancy — a missing move, an extra move, a wrong stacking rule — shifts a count. (Journal H2-2026-09-09-suite-rerun-01.)

2. Reference-engine agreement. The engine passes nokamute’s UHP conformance harness (21/21) and, more stringently, differential fuzzing plays seeded random games while asserting per-ply *set equality* of legal moves against two independent reference engines — MzingaEngine v0.16.0 (the UHP reference implementation) and nokamute 1.0.3: 27,829 positions at the session’s fixed seed, 200/100 games per type nightly. Two engines with independent codebases agreeing on every legal-move set bounds the chance of a shared rules misreading. (Journal H2-2026-09-09-suite-rerun-01.)

3. Hand-annotated critical corpus. Reference agreement cannot catch a misreading shared by the community’s engines, so 30 critical positions were annotated *by hand from the publisher’s rules* (Gen42 rulesheet and Pillbug sheet, World Hive Tournaments FAQ; tournament opening rule declared as a convention) — sliding and gates, beetle stacking and the above-ground beetle gate, stack colour for placement, One-Hive cut points and rings, win/draw termination including the simultaneous-surround draw, forced pass, and Pillbug stun as a kernel guard. Expectations were committed before the first engine run; the runner compares full (piece, destination) move sets over UHP. First run: 29/30, with the single disagreement resolved *against the corpus* — a setup line violated the One-Hive transit rule the engine correctly enforces — and 30/30 after the correction; the investigation is journaled either way (invariant 2). Declared limit: the annotations have not yet had an external Hive-literate review. (Journal H2-2026-09-09-corpus-run-01.)

4. Random invariant sessions. Seeded random games across all 8 game types check, after every transition: every generated move is accepted by the apply path (and undo restores the exact GameString), the hive remains connected, serialise→deserialise through the UHP GameString

reproduces the position, result, and exact valid-move set, and pass is accepted exactly when no move exists. 10,665,686 generated moves were applied and undone over 161,546 plies (deep session) with zero violations; a smaller session runs in the standard suite on every build. (Journal H2-2026-09-09-random-invariants-01.)

Scope and honest gaps

Code coverage is not claimed as correctness evidence (plan ch. 4). The lines above bound different failure modes (exhaustive-at-depth, agreement, rules-text fidelity, invariant stability) but none proves full-game perfection. Known issues found during profiling and deferred to the pipeline phase, recorded as findings rather than silently fixed: the *prior-work* self-play generators map their 300-ply cap to a draw (invariant 7 forbids this in any new-study measurement code), and the Rust ONNX CoreML execution provider currently crashes on the study machine while the same model runs on CoreML through Python onnxruntime at 2.62 ms/eval. (Journal H2-2026-09-09-throughput-profile-01.)

Cross-language data-path check

The grid arm's Rust plane encoder and the Python training-side decoder are kept byte-identical, verified by a golden-file crosscheck (240 positions, exact agreement) that runs in the nightly script. (Journal H2-2026-09-09-suite-rerun-01.)

Method — Baseline opponents and evaluation population (report section draft)

Drafted 2026-09-09 at the H3 close (invariant 14). English working master; French copy at the report milestone (D-006). Numbers trace to journal H3-2026-09-09-baselines-01; claims registered in `claims.md`.

Why a frozen population

Both architectures are scored by mean result against a **fixed opponent population frozen before any training run** (protocol §4): win = 1, draw = 0.5, loss = 0, truncations excluded and reported as their own rate. A population fixed in advance is what makes scores comparable across arms, seeds and budgets; touching an opponent afterwards would silently move the measuring stick, so the population is under freeze discipline — it was frozen on 2026-09-09 with explicit approval (D-017: agent configs and weight file by sha256, engine code commit) and no member may be added, removed or retuned since. If an Elo-style number is ever quoted it is descriptive and relative to this population only.

The three agents

All three run on the validated engine (the same rules kernel as both study arms) over UHP, fully seeded:

- **B-RND — legal-random.** Uniform over legal moves; deterministic given (seed, game history). The floor and score anchor.
- **B-HEU — documented heuristic.** Greedy one-ply argmax of a handcrafted evaluation (queen-safety dominant, mobility-as-material, small tempo terms — every feature and weight tabulated in `docs/baselines.md`), plus the searcher's quiescence on enemy-queen-targeting moves. The weights were inherited unchanged from the prior engine and are pinned by an automated test: no

tuning occurred during H3 and any later change breaks the suite. This closes the tune-after-observing-results path by construction.

- **B-MCTS — search without network.** PUCT MCTS with uniform priors and the same handcrafted evaluation (tanh-squashed) as leaf value; no exploration noise at evaluation. Budget: 6400 simulations per decision — chosen from measured cost (≈ 27 ms/decision single-thread on the study machine), not from the plan’s placeholder figures.

Search verification before use

The MCTS baseline was verified on a hand-annotated tactical set written from the publisher’s rules (same discipline as the engine-validation corpus, committed before any run): mate-in-1 by perimeter walk and by grasshopper jump, from both colours, and an avoid-self-surround case — 5/5 at 400, 1600 and 6400 simulations. Value signs under player alternation are pinned by automated tests at three levels: the evaluation negates exactly when the side to move flips; alpha-beta scores a mate-in-1 above the mate threshold for the mover, whichever colour moves; the MCTS root value is strongly positive for the winning mover, whichever colour moves.

Characterisation

100 paired, colour-swapped games per pairing from common seeded random openings, truncations reported separately (none occurred in 300 games):

Pairing	W/D/L	Score	Elo (descriptive)
B-HEU vs B-RND	100/0/0	100.0%	$\approx +2400$
B-MCTS vs B-RND	99/1/0	99.5%	+920 [+730, +1200]
B-MCTS vs B-HEU	23/29/48	37.5%	-89 [-150, -32]

One ordering surprised us and is reported as found: the 6400-sim search sits *below* the one-ply heuristic. Diagnosis (supported by a 4×-budget probe reaching 56.2%): with uniform priors, 6400 simulations spread over Hive’s ~ 60 -move branching yield an effectively shallow search, while the greedy agent’s queen-targeting quiescence is tactically sharp — an echo of the literature, where plain search and heuristics are strong in Hive (de Goede et al. 2022; Kampert et al. 2021). No retuning followed the observation; the population deliberately spans a floor plus two mid-band opponents of different styles roughly 90 Elo apart.

Limits

Characterisation volume is the pilot’s 100 games per pairing with an unpaired-approximation interval; the tactical set (5 cases) and its annotations are unreviewed by an external Hive-literate reader — the same declared limit as the engine-validation corpus.

Method — Policy-value pipeline and pre-training checks (report section draft)

Drafted 2026-09-10 at the H4 close (invariant 14). English working master; French copy at the report milestone (D-006). Numbers trace to journals H4-2026-09-09-coreml-fix-01 and H4-2026-09-10-pilot-01; claims registered in `claims.md`.

The shared action decoder

Both architectures emit a policy over the same action space — a move is (side-relative piece slot, destination cell) plus pass — with identical legal-set masking, normalisation over exactly the legal set, identical MCTS visit-distribution targets, and a deterministic tiebreak (`docs/action-decoder.md`, D-015). The grid arm materialises this space as a 28,673-way tensor over its 32×32 frame; the graph arm (H5) scores the identical candidate pairs per position with empty destination cells as first-class nodes. Nothing about the action interface differs between arms; only the state encoder and scoring function do — this is the study’s central confounder control.

Data discipline

Self-play records are versioned and self-describing: each carries the generating model’s stamp (generation + net hash), the position, the visit-distribution target, the legal-move index list that enables masked training, and a four-valued outcome — win, draw, loss, or **truncated** (a ply-cap game is never recorded as a draw; truncated records are excluded from the value loss). A per-run manifest ties every shard to its model, and the run layout separates `selfplay/` (training inputs), `checkpoints/`, and `eval/` (never training input), enforced by an audit script.

Seven pre-training checks

All automated, run as one command against real shards before any training is trusted: (1) the masked softmax puts exactly zero mass on illegal actions and sums to one over legal actions, on real positions through a real forward pass; (2) `encode→id→decode` is the identity on all legal moves across game types (and stored targets are always legal); (3) outcomes are correct for the player to move, both colours, with truncation distinct; (4) a fixed tiny batch overfits to near-perfect fit — policy `argmax` 15/15, value 15/15, residual KL 0.09 against soft targets (the criterion is KL to the target-entropy floor: soft visit distributions have irreducible entropy, so “loss → 0” is the wrong test); (5) save/resume reproduces bitwise-identical model and optimizer state, with the interruption preserving the planned LR schedule; (6) exploration noise is structurally evaluation-impossible (the eval path’s defaults carry $\epsilon = 0$, asserted by test; settings pinned by config + decision D-019); (7) no evaluation game can reach training, by layout and audit.

Small-budget pilot

One full generation from a seeded random initialisation (1.44M-param grid net, 300 games at 128/32 simulations with playout-cap randomization): 17,237 positions, 56.7% of gen-0 games truncated (reported, not folded into draws), training non-degenerate (policy above chance, value above base rate, losses decreasing), and independent evaluation against the frozen population under pinned settings: **100–0 vs legal-random**, 3.3% vs the documented heuristic, 3.3% vs 6400-sim MCTS. The diagnosis is clean and follows the mandated order (rules, signs, search, data — the first three independently validated in H2/H3): after one generation the gap to strong opponents is a data and iteration gap, and the indicated lever is more generations, not more capacity. Budget confirmation: worst-case (gen-0) generation cost is ≈ 12 s/game wall at 4 threads, falling toward ≈ 3 s/game as play sharpens with a trained net.

Limits

Single seed end-to-end at pilot scale; 30 evaluation games per opponent; only the grid arm exists yet (the graph arm arrives in H5 against the same decoder contract); gen-1+ iteration exercised structurally, not run.

Method — The two representations (report section draft)

Drafted 2026-09-10 at the H5 close (invariant 14). English working master; French copy at the report milestone (D-006). Numbers trace to journals H5-2026-09-10-encoders-01 and H5-2026-09-10-graph-wiring-01; claims registered in `claims.md`.

What varies, and what provably does not

The study's independent variable is the state representation and its network body — nothing else. Both arms share the engine, the search, the frozen opponents and pinned evaluation settings, the self-play records, the training targets and outcome conventions, the action space with its legal-set normalisation, and the training-loop machinery. Each shared element is enforced rather than asserted: the decoder is one contract with identical masking; the records are one format; each encoder is pinned by a cross-language golden test (240 positions for the grid planes, 160 for the graph tensors, byte-exact, run nightly); and both arms evaluate through the same Rust MCTS via ONNX exports.

The grid arm

The baseline representation embeds the position in a fixed 32×32 frame: BFS-unwrapped from the engine's torus, bounding-box-centred, with 77 feature planes (piece/owner/type/height jointly, pins, stun, placement regions, and global scalars as constant planes). A 28-piece hive plus its full ring of candidate destinations fits the frame by construction; that argument is enforced by an always-on assert and extremal tests (28-piece straight lines: complete, alias-free mapping) — no piece can vanish silently, in release builds included. The network is a KataGo-lite ResNet (1.44 M parameters) with a flat (piece-slot $\times$ frame-cell) policy head.

The graph arm

The graph representation is coordinate-free: nodes are the cells of the candidate set — every occupied cell and every empty 1-ring cell, so each scorable destination is a first-class node; edges are typed by the six hex directions; node features carry the stack level-by-level (owner and bug type per level), pins, stun, and placement regions; reserves, side-to-move, ply and game type enter as a global vector. Pieces are not separate nodes: the shared decoder addresses moves as (piece slot, destination cell), so piece identity enters the policy through a slot embedding and the piece's standing node — a documented design choice, not an accident. The network is a small relational message-passing net (direction-specific weights, global-pooling bias, masked pooling for the value head, per-candidate move scoring) at 1.47 M parameters — a +2.1% capacity difference, reported.

A graph network grants no invariance or rule equivalence for free. The encoding contains no absolute coordinates, but the learned function is not thereby translation- or rotation-invariant, and no rule of Hive is built in; any such claim in this study is measured, never assumed.

Measured cost asymmetries (reported, not equalised)

On the study machine, the best available inference path differs per arm: the convolutional grid net runs on the CoreML accelerator at 2.62 ms/evaluation, while the gather-heavy graph net runs fastest on CPU at 3.67 ms (CoreML is slower for it) — a $\approx 1.4 \times$ per-decision cost against the graph arm. Training shows the reverse pattern per device (CPU favours graph $3.5 \times$; the MPS path used for training favours grid

$\approx 2\times$). These are genuine hardware interactions of the representations; the protocol’s two budget readings (same-examples and same-wall-clock) charge them honestly rather than hiding them.

Symmetry augmentation

Excluded from the full method for both arms (D-023): exclusion makes the identical-data-protocol rule trivially true, avoids implementing hex symmetries differently per representation, and leaves augmentation as a clean additive ablation question.

Limits

Costs are single-machine; the wiring validation trained the graph arm at smoke scale only (its learning profile on gen-0 data matched the grid arm’s, as expected for matched arms on shared data — not a comparison result); the controlled comparison itself is H6, under protocol v1.0.

Chapter 5 — Experimental protocol (booklet draft, 2026-09-26)

Source of authority: the frozen protocol v1.0 (D-020, sha256 f340a6b6...aefeb5) and the freeze/pin decisions D-017, D-019, D-025, D-026, D-027. This chapter restates them for the reader; the frozen document prevails on any divergence.

5.1 Pre-registration discipline

Everything that could bias the comparison was frozen before any comparison run existed, in this order: opponent population (D-017), protocol with its rejection rule and measured budgets (D-020), fixed shared openings (D-025); evaluation settings were pinned by config and test (D-019). The same-wall-clock cutoff T^* was computed by a rule stated in the pre-registered matrix — the median full-run wall-clock of the grid arm’s three runs — from grid clocks alone, before any cross-arm number was assembled. No frozen artifact was modified at any point; the repository history documents this.

5.2 Matrix, seeds, hardware, budgets

2 architectures $\times$ 3 training seeds (1–3; disjoint seed spaces), 10 generations $\times$ 500 self-play games per run at 128 full / 32 cheap simulations per decision (measured choices, not the plan’s placeholder values), 300-ply cap. One machine (Apple M1 Pro, 10 cores, 16 GB), four worker threads per run, runs sequential; each arm uses its measured best inference provider (grid: CoreML at 2.62 ms/eval; graph: CPU at 3.67 ms/eval) — a real hardware interaction reported and charged, not equalised away. Every run logs wall-clock per generation, states, and simulations; checkpoints are kept at every generation.

5.3 Opponents, openings, pairing

The evaluation population is frozen: seeded legal-random (B-RND), a documented fixed-weight heuristic (B-HEU; weights hash-pinned and test-enforced), and 6400-simulation no-network MCTS (B-MCTS). The gen-19 checkpoint of the prior demonstration loop is excluded (D-016/17). Openings are 250 pre-generated, unique, legal 4-ply openings (seeded generator, content hash recorded); every match plays pair i on opening line i with colours swapped, so all arms, seeds and opponents face identical opening/colour schedules (verified by schedule-hash test).

5.4 Metrics, intervals, exclusions

Primary metric: mean score (win 1 / draw 0.5 / loss 0) against the population, per (seed, opponent) cell, **excluding truncated games**, whose rate is always reported separately, with a truncations-as-0.5 sensitivity column and cap-treatment bounds (truncations as losses and as wins) bracketing any alternative cap. The resampled unit is the **seed**: cells aggregate games first; intervals are percentile bootstrap (10,000 resamples) over the three seeds; the arm contrast bootstraps both arms' seed sets independently. Games are never pooled as i.i.d.; Elo, where printed by tooling, is descriptive only. Checkpoint selection is mechanical: generation-10 checkpoints for the same-examples reading; the last checkpoint completed at $\leq T^*$ for the same-wall-clock reading. **Exclusion criteria**: none were needed — no run failed, none was excluded, and a bad score is not an exclusion criterion by protocol.

5.5 RQ → experiment → result matrix

RQ	Experiment	Result artifact
RQ-H1: does the graph arm beat the grid arm at comparable budget?	The H6 matrix under both readings vs the frozen population	Per-seed score tables + arm contrast with seed-bootstrap intervals (results-same-examples / results-same-wallclock / results-arm-difference); verdict via the frozen rejection rule
RQ-H2 (secondary): per-example vs per-hour reading	The same campaign read at generation-10 vs T^* -checkpoints	Score-vs-time curves (fig1); score/cost table (fig2)
RQ-H3 (secondary): what do graph components contribute?	Two one-component ablations at parity (A1 edge typing; A2 global pooling, substitution D-028)	Ablation table H-T3 (results/ablations/); A1' supplementary (two-component, clearly labeled) if run

5.6 Reproduction

Every table and figure regenerates from scripts over raw per-game records (`scripts/make_results.py`, `make_figures.py`); every run's generator settings, seeds and model stamps live in per-run manifests; commands, configs and seeds are annexed. The minimal reproduction scenario (annex) replays a single recorded evaluation game deterministically and regenerates the result tables from the shipped records.

Results — The controlled comparison (report section draft)

Drafted 2026-09-19 at the H6 close (invariant 14). English working master; French copy at the report milestone (D-006). Every number traces to journals H6-2026-09-19-comparison-01 (3-seed analysis) and H6-2026-10-09-5seed-final-01 (final 5-seed analysis) and regenerates from `scripts/make_results.py` / `make_figures.py` over the raw per-game records; claims registered in `claims.md`.

The pre-registered question and its answer

The protocol (frozen at v1.0 before any comparison run) asked: at comparable training budget, does the graph architecture learn a better policy than the grid architecture for base-game Hive? It fixed, in advance, what would reject the hypothesis: no seed-consistent graph advantage under both budget readings, with intervals excluding a meaningful graph advantage.

That is what happened. H1 is rejected. Under the same-examples reading (both arms, 10 generations $\times$ 500 games) and the same-wall-clock reading ($T^* = 18.77$ h, computed by a pre-registered rule before any cross-arm number existed), the graph arm scored lower against two of the three frozen opponents and no better against the third, across **five seeds per arm** (seeds 4–5 were added after the three-seed analysis, symmetrically and under a pre-commitment to use all seeds regardless of direction, D-031; the three-seed analysis reached the same verdict):

graph – grid	Same-examples	Same-wall-clock
vs legal-random	–0.169 [–0.272, –0.062]	–0.161 [–0.278, –0.048]
vs heuristic	–0.064 [–0.111, –0.017]	–0.059 [–0.087, –0.029]
vs 6400-sim MCTS	–0.009 [–0.055, +0.028]	–0.018 [–0.066, +0.025]

(Seed-level means; bootstrap 95% over five seeds per arm; per-seed values in the results tables — no best-seed reporting anywhere. The largest upper bound across all six contrasts is +0.028.)

The wall-clock reading compounds the result: the graph arm’s measured cost was $2.0\times$ per run (35.7 vs 18.2 h), so at equal hours it completes only 4–5 of 10 generations — a deficit the per-example reading already shows and equal time only widens.

Truncation, reported separately and stress-tested

The clearest behavioural difference is not a score but an outcome category: against legal-random, the graph arm truncated 20–57% of its games at the 300-ply cap across the original seeds and 23–44% in the extension seeds (grid: 0–1%, with one extension seed at 11%) — winning material and then failing to convert (fig. 4, F1). Because truncation was defined as its own outcome from the start, this pathology is visible rather than laundered into draws. The cap value cannot rescue the hypothesis: even scoring every truncated game as a graph win — an upper bound on any larger cap — leaves the graph arm behind on the random opponent under both readings (–0.072 / –0.058).

What this does and does not show

It shows: for a capacity-matched (+2.1%), simple relational message-passing network sharing every other component with the grid arm — rules, search, decoder, records, training conventions, frozen opponents, frozen openings, pinned evaluation — grid-plane encoding learned more per example AND per hour at this small budget, and the graph arm’s weakness concentrates in local tactical conversion, matching the local-vs-long-range asymmetry reported for Hex under DQN (Keller et al. 2023) now observed under matched AlphaZero-style self-play in a frameless, stacking game.

It does not show: anything about graph representations at larger budgets, other graph architectures, or other games; nor that the arms would not reorder with more generations (10 is early-regime; all scores against the strong baselines remain low). Five seeds per arm bound the statistics; the rejection is seed-consistent, and the extension seeds (added under pre-commitment) tightened four of the six intervals.

Costs (both denominations, per plan ch. 6)

Fig. 2: parameters 1.44M vs 1.47M; best-provider inference 2.62 ms (CoreML) vs 3.67 ms (CPU); self-play 13.1 vs 25.7 s/game; population score 0.412 vs 0.327 (same-examples), 0.402 vs 0.321 (same-wall-clock).

Provenance

Population frozen 2026-09-09 (D-017); protocol frozen 2026-09-10 (D-020); openings frozen 2026-09-10 (D-025) — all before any comparison run. Campaign approved and launched 2026-09-10 (D-026); T* computed 2026-09-16 from grid wall-clocks only; no opponent, opening, or protocol touch occurred at any point. Negative result retained and reported per invariant 5 and protocol §1: the work does not need H1 confirmed to count.

The ablations (H-T3)

Two single-component ablations against the graph full method, three seeds each at full budget parity, plus one labeled supplement. Removing the direction-typed edge relations (A1, naive adjacency) destroyed trainability outright — NaN divergence in generation 0 for every seed — so typed edges carry, at minimum, the arm's optimization stability. Removing the global-pooling bias (A2) produced a null: score changes of -0.001 [-0.170 , $+0.165$], -0.007 [-0.043 , $+0.030$] and -0.013 [-0.043 , $+0.013$] against the three opponents, with the failure modes unchanged. The supplementary A1' (untyped edges plus gradient clipping — an explicitly two-component variant, never attributed to typing alone) trains stably and scores within the full arm's band against all three opponents, suggesting the typed relations' measurable contribution at this scale is concentrated in optimization stability rather than final strength — a statement that inherits the clip confound and is phrased accordingly wherever it appears.

Chapter 7 — Discussion and threats to validity (booklet draft, 2026-09-26)

Four-way split per plan ch. 21. Grounded in the claims register.

7.1 Reading the result

H1 was rejected exactly as the frozen protocol defined rejection. Three observations give the result its texture. First, the deficit is opponent-shaped: largest against legal-random, clear against the heuristic, absent against the search-heavy B-MCTS — consistent with a graph arm that is weakest at *local tactical conversion* rather than at positional judgment. Second, the truncation channel carries the mechanism: the graph arm wins material against random and then fails to close (20–57% capped games; fig4-F1), a pattern invisible in studies that fold caps into draws. Third, the ablations sharpen the reading: removing geometric edge typing does not merely weaken the graph arm — at parity settings it destroys trainability outright (NaN divergence, generation 0, 3/3 seeds), so the typed relations carry at minimum the optimization stability of the whole arm — while the second ablation found the global-pooling bias fully dispensable (score changes $\approx 0.00 \pm 0.17$, -0.01 ± 0.04 , -0.01 ± 0.03 against the three opponents; failure modes unchanged), localising the arm's distinctive machinery in the directional relations, not the pooling. This contrasts instructively with the positive chess result of Rigaux & Kashima (2024, NeurIPS) and Keller et al.'s (2023) Hex asymmetry: our result does not contradict them — it bounds where their optimism transfers, and the comparison of evidence standards matters: the chess result rests on a single training run per model with intervals covering Elo estimation only, whereas the present rejection is seed-consistent across five runs per arm under two pre-registered budget readings. Hive's tactics are

dominated by short-range surround geometry, the regime Keller et al. found CNNs stronger in; at small budgets that regime decides games.

7.2 Internal validity (bugs, comparability)

The engine is validated independently of learning (perft to depth 6+ against published tables, 21/21 UHP conformance, 27,829-position differential agreement with two reference engines, a rules-derived hand-annotated corpus, 10.9M-transition invariant sessions). Both encoders are pinned byte-exactly against cross-language goldens run nightly. Arms share the decoder, records, losses, budgets and search; capacity differs by +2.1% (reported). Residual risks: the graph architecture is ONE point in design space — a stronger GNN might behave differently (we claim nothing beyond this net); tooling defects found during the study (an arena record-loss path on all-truncated matches; a silent NaN passage in ablation training) were caught by the verification discipline, fixed, and audited to have left campaign data untouched — but they illustrate that harness error, not chance, is the dominant failure mode at this scale.

7.3 Measurement validity (opponents, truncation)

The population is three fixed opponents spanning floor-to-mid strength; all trained arms still lose heavily to the two strong baselines, so the comparison lives in a low-score regime where differences against B-MCTS are hard to resolve. Scores are relative to THIS population — no universal strength claim is made. Truncation is handled as its own outcome with sensitivity bounds; the 300-ply cap itself cannot reverse the verdict (truncations-as-wins bound stays negative), but the high graph truncation rates mean its random-opponent score is measured on fewer decided games (43–80 per seed).

7.4 Statistical validity (seeds, dependencies)

Five seeds per arm bound the statistics — the pre-committed extension from three to five (D-031) tightened four of the six intervals and absorbed the most graph-favourable seeds observed without changing the verdict; the B-MCTS contrast remains indistinguishable from zero under both readings. The rejection does not rest on a single interval: it rests on seed-consistency across two opponents and two readings simultaneously, plus bounds analysis. Openings are shared across arms (pairing respected in the design); the arm contrast bootstraps seeds, not games; no game-level pseudo-replication enters any interval.

7.5 External validity (variant, hardware, budget)

One variant (base Hive), one machine, one small budget (10 generations $\times$ 500 games; ~18–48 h/run), early-regime self-play throughout. The provider asymmetry (CoreML favours the convolutional net; the gather-heavy GNN runs fastest on CPU) is a genuine property of the deployment hardware, reported and charged — on different accelerators the wall-clock reading could shift. Nothing here generalises to other games, larger budgets, or richer graph architectures; the study answers its pre-registered question inside its pre-registered perimeter, and the negative answer is the product: encoding choice matters (as AZ-Hive found), and for Hive at small budget it favours the grid.

Chapter 8 — Conclusion (booklet draft, 2026-10-09)

No new results here, per the plan.

Within the tested perimeter — base-game Hive, one capacity-matched simple relational message-passing network against one grid CNN, ten generations of small-budget self-play, five seeds per arm, a frozen three-opponent population — **the answer to RQ-H1 is no**: the graph representation did not learn a better policy than the grid representation, under the same-examples reading or the same-wall-clock reading, and the pre-registered rejection rule fired exactly as frozen. The deficit is largest where Hive is most tactical, the graph arm pays twice the wall-clock, and its failure mode — winning material without converting — is visible precisely because truncation was never folded into draws. The result does not contradict the positive graph findings in Hex and chess; it bounds them: where short-range tactics decide games and budgets are small, frame artifacts are cheaper than framelessness.

Two follow-ups are motivated by the data rather than by hope. First, **the conversion pathology is a targetable defect**: the graph arm’s value head learns respectably while its policy fails at forcing sequences, suggesting an experiment on search-time remedies (deeper evaluation budgets in won positions, or auxiliary targets for forcing moves) under the same frozen evaluation — a one-component change to the shared pipeline, applicable to both arms. Second, **the stability finding deserves isolation**: A1/A1’ showed direction-typed relations matter mostly for optimization at this scale; a controlled study of normalisation and gradient-scale choices for relation-shared graph layers could decouple trainability from representational content — and would say whether naive adjacency, properly stabilised, is genuinely sufficient for Hive.

The broader method stands regardless of the sign of the result: pre-registration with frozen artifacts, dual budget readings, truncation as an outcome, seed-level inference, and write-as-you-go reporting turned a negative answer into a usable scientific object on a single laptop.

Bibliography and webographie (generated — plan ch. 27)

Generated by `scripts/make_bibliography.py` from the reading notes; every entry was actually consulted (its note is the evidence) and no entry appears in both sections.

Bibliography

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, Marc G. Bellemare. “Deep Reinforcement Learning at the Edge of the Statistical Precipice. NeurIPS 2021 (peer-reviewed; Outstanding Paper Award); arXiv:2108.13264. arXiv:2108.13264 (v1 Aug 30, 2021; v4 Jan 5, 2022), DOI 10.48550/arXiv.2108.13264; NeurIPS 2021 proceedings. Code: google-research/rliable (pip package rliable). (*note: docs/reading/agarwal-2021-precipice.md, read 2026-09-09*)
- Shai Ben-Assayag, Ran El-Yaniv. “Train on Small, Play the Large: Scaling Up Board Games with AlphaZero and GNN. arXiv preprint 2021 (arXiv:2107.08387 [cs.LG], v1 submitted 18 Jul 2021; no journal/conference version found as of 2026-09-26). arXiv:2107.08387v1 (18 Jul 2021), DOI 10.48550/arXiv.2107.08387; no code commit (code unreleased) (*note: docs/reading/benassayag-2021-scalable-alphazero.md, read 2026-09-26*)
- Tristan Cazenave, Yen-Chi Chen, Guan-Wei Chen, Shi-Yu Chen, Xian-Dong Chiu, Julien Dehos, Maria Elsa, Qucheng Gong, Hengyuan Hu, Vasil Khalidov, Cheng-Ling Li, Hsin-I Lin, Yu-Jin Lin, Xavier Martinet, Vegard Mella, Jeremy Rapin, Baptiste Roziere, Gabriel Synnaeve, Fabien Teytaud, Olivier Teytaud, Shi-Cheng Ye, Yi-Jun Ye, Shi-Jim Yen, Sergey Zagoruyko. “Polygames: Improved Zero Learning. ICGA Journal 42(4), pp. 244-256, 2020 (issue published Jan 2021); arXiv:2001.09832 (Jan 2020). DOI 10.3233/ICG-200157; arXiv:2001.09832; code repo archived

2022-03-02 (no pinned release cited) (note: *docs/reading/cazenave-2020-polygames.md*, read 2026-09-09)

- Danilo de Goede, Duncan Kampert, Ana Lucia Varbanescu. “The Cost of Reinforcement Learning for Game Engines: The AZ-Hive Case-study. ICPE 2022 (13th ACM/SPEC International Conference on Performance Engineering, Beijing), pp. 145-152. DOI 10.1145/3489525.3511685; open PDF at https://research.spec.org/icpe_proceedings/2022/proceedings/p145.pdf; BeeKeeper engine repo commit not pinned in paper (note: *docs/reading/degoede-2022-azhive.md*, read 2026-09-09)
- William L. Hamilton, Rex Ying, Jure Leskovec. Inductive Representation Learning on Large Graphs. NeurIPS 2017 (peer-reviewed; NIPS at the time). arXiv:1706.02216.. arXiv:1706.02216 (v4, 2018-09-10), DOI 10.48550/arXiv.1706.02216; NeurIPS 2017 proceedings. Code: github.com/williamleif/GraphSAGE (no specific release pinned). (note: *docs/reading/hamilton-2017-graphsage.md*, read 2026-09-09)
- Andy L. Jones. “Scaling Scaling Laws with Board Games. arXiv preprint 2021 (arXiv:2104.03113; v1 Apr 7, 2021, v2 Apr 15, 2021; not peer-reviewed). arXiv:2104.03113v2, DOI 10.48550/arXiv.2104.03113. Code/data: boardlaw project (andyljones.com/boardlaw, github.com/andyljones/boardlaw). (note: *docs/reading/jones-2021-scaling.md*, read 2026-09-09)
- Duncan Kampert, Ana-Lucia Varbanescu, Matthias Müller-Brockhausen, Aske Plaat. “Mimicking the Human Approach in the Game of Hive.” (preprint circulated as “Better AI for Hive: Mimicking human game-play strategies”) IEEE SSCI 2021 (IEEE Symposium Series on Computational Intelligence, Orlando, Dec 2021). IEEE Xplore document 9659999 (<https://ieeexplore.ieee.org/document/9659999/>); preprint PDF: https://liacs.leidenuniv.nl/~plaata1/papers/IEEE_Conference_Hive_D_Kampert.pdf; dblp: conf/ssci/KampertVMP21 (note: *docs/reading/kampert-2021-mimicking-hive.md*, read 2026-09-09)
- Yannik Keller, Jannis Blüml, Gopika Sudhakaran, Kristian Kersting. From Images to Connections: Can DQN with GNNs learn the Strategic Game of Hex? arXiv preprint 2023 (arXiv:2311.13414; submitted to ICLR via OpenReview id dYaeDrazj5, not listed as accepted — treat as non-peer-reviewed preprint).. arXiv:2311.13414 (2023-11-22), <https://arxiv.org/abs/2311.13414>; OpenReview forum dYaeDrazj5. Code: github.com/yannikkellerde/GNN_Hex (no release pinned). (note: *docs/reading/keller-2023-graphdqn-hex.md*, read 2026-09-09)
- Tomas Rigaux, Hisashi Kashima. “Enhancing Chess Reinforcement Learning with Graph Representation. NeurIPS 2024 (Advances in Neural Information Processing Systems 37, main conference track); arXiv:2410.23753. arXiv:2410.23753v1 (31 Oct 2024), DOI 10.48550/arXiv.2410.23753; NeurIPS proceedings DOI 10.52202/079017-0006; code repo not pinned to a commit in the paper (note: *docs/reading/rigaux-2024-chess-graph-rl.md*, read 2026-09-26)
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, Demis Hassabis. “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm.” Also published as: “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” Science 362(6419):1140-1144, 2018. arXiv preprint 2017 (arXiv:1712.01815, v1 Dec 5, 2017); peer-reviewed version in Science 2018 (vol. 362, issue 6419, pp. 1140-1144). arXiv:1712.01815v1, DOI 10.48550/arXiv.1712.01815; Science version DOI 10.1126/science.aar6404. No official code release. (note: *docs/reading/silver-2017-alphazero.md*, read 2026-09-09)
- Elise van der Pol, Daniel E. Worrall, Herke van Hoof, Frans A. Oliehoek, Max Welling. MDP Homomorphic Networks: Group Symmetries in Reinforcement Learning. NeurIPS 2020 (peer-reviewed). arXiv:2006.16908.. arXiv:2006.16908 (v2, 2021-01-20),

<https://arxiv.org/abs/2006.16908>; NeurIPS 2020 proceedings
<https://papers.nips.cc/paper/2020/hash/2be5f9c2e3620eb73c2972d7552b6cb5-Abstract.html>.
Code: github.com/ElisevanderPol/mdp-homomorphic-networks (no release pinned). (*note: docs/reading/vanderpol-2020-mdp-homomorphic.md, read 2026-09-09*)

- Oriol Vinyals, Meire Fortunato, Navdeep Jaitly. Pointer Networks. NeurIPS (NIPS) 2015, Advances in Neural Information Processing Systems 28 (peer-reviewed). arXiv:1506.03134.. arXiv:1506.03134 (v2, 2017-01-02), <https://arxiv.org/abs/1506.03134>; NIPS 2015 proceedings https://proceedings.neurips.cc/paper_files/paper/2015/hash/29921001f2fo4bd3baee84a12e98098-Abstract.html. (*note: docs/reading/vinyals-2015-pointer-networks.md, read 2026-09-09*)
- David J. Wu. “Accelerating Self-Play Learning in Go. arXiv preprint (arXiv:1902.10565; v1 Feb 27, 2019, v5 Nov 9, 2020); presented at the AAAI-20 Workshop on Reinforcement Learning in Games (not a full peer-reviewed proceedings paper). arXiv:1902.10565v5, DOI 10.48550/arXiv.1902.10565. Code: github.com/lightvector/KataGo (our reference reading: v5 paper; cite a specific release tag if we import any technique implementation). (*note: docs/reading/wu-2020-katago.md, read 2026-09-09*)

Webographie

- Jan Pfeifer (GitHub user janpfeifer). “hiveGo — Go Implementation of Hive Game.” GitHub repository. GitHub repository, 2018–2026 (created 2018-08-24, originally TensorFlow; refreshed 2025 onto GoMLX; last push 2026-08-20). main branch, commit d6ff95418d2b (2026-08-20, “Updated main.wasm version.”); <https://github.com/janpfeifer/hiveGo>; consulted 2026-09-26 (*note: docs/reading/pfeifer-2025-hivego.md, read 2026-09-26*)

Result tables and figures (generated)

Pulled verbatim at assembly time from results/ and paper/figures/ — regenerate via scripts/make_results.py and scripts/make_figures.py.

H6 results — same-examples reading

Score = mean over non-truncated games (win 1 / draw 0.5 / loss 0); trunc = truncation rate; sens = truncations scored 0.5.

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
grid	s1	0.995 / 0%	0.150 / 0%	0.125 / 0%
grid	s2	0.975 / 1%	0.080 / 0%	0.125 / 0%
grid	s3	0.990 / 1%	0.190 / 0%	0.075 / 0%
grid	s4	0.949 / 11%	0.105 / 0%	0.085 / 0%
grid	s5	0.995 / 0%	0.120 / 0%	0.210 / 0%
graph	s1	0.733 / 57%	0.025 / 0%	0.115 / 0%
graph	s2	0.981 / 20%	0.050 / 0%	0.125 / 0%
graph	s3	0.722 / 55%	0.090 / 0%	0.100 / 0%
graph	s4	0.688 / 44%	0.125 / 0%	0.120 / 0%

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
graph	s5	0.935 / 23%	0.035 / 0%	0.115 / 0%

Seed-level means (bootstrap 95%, unit = seed)

- grid vs B-RND: mean 0.981 [0.964, 0.994] (seeds: 0.995, 0.975, 0.990, 0.949, 0.995)
- grid vs B-HEU: mean 0.129 [0.098, 0.165] (seeds: 0.150, 0.080, 0.190, 0.105, 0.120)
- grid vs B-MCTS: mean 0.124 [0.087, 0.168] (seeds: 0.125, 0.125, 0.075, 0.085, 0.210)
- graph vs B-RND: mean 0.812 [0.710, 0.920] (seeds: 0.733, 0.981, 0.722, 0.688, 0.935)
- graph vs B-HEU: mean 0.065 [0.034, 0.100] (seeds: 0.025, 0.050, 0.090, 0.125, 0.035)
- graph vs B-MCTS: mean 0.115 [0.107, 0.121] (seeds: 0.115, 0.125, 0.100, 0.120, 0.115)

H6 results — same-wallclock reading

Score = mean over non-truncated games (win 1 / draw 0.5 / loss 0); trunc = truncation rate; sens = truncations scored 0.5.

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
grid	s1	0.995 / 0%	0.150 / 0%	0.125 / 0%
grid	s2	0.975 / 1%	0.080 / 0%	0.125 / 0%
grid	s3	0.939 / 1%	0.145 / 0%	0.080 / 0%
grid	s4	0.949 / 11%	0.105 / 0%	0.085 / 0%
grid	s5	0.995 / 0%	0.120 / 0%	0.210 / 0%
graph	s1	0.798 / 58%	0.045 / 0%	0.075 / 0%
graph	s2	0.926 / 39%	0.055 / 0%	0.110 / 0%
graph	s3	0.713 / 53%	0.060 / 0%	0.105 / 0%
graph	s4	0.631 / 39%	0.100 / 0%	0.150 / 0%
graph	s5	0.980 / 24%	0.045 / 0%	0.095 / 0%

Seed-level means (bootstrap 95%, unit = seed)

- grid vs B-RND: mean 0.971 [0.950, 0.991] (seeds: 0.995, 0.975, 0.939, 0.949, 0.995)
- grid vs B-HEU: mean 0.120 [0.098, 0.142] (seeds: 0.150, 0.080, 0.145, 0.105, 0.120)
- grid vs B-MCTS: mean 0.125 [0.090, 0.168] (seeds: 0.125, 0.125, 0.080, 0.085, 0.210)
- graph vs B-RND: mean 0.810 [0.697, 0.922] (seeds: 0.798, 0.926, 0.713, 0.631, 0.980)
- graph vs B-HEU: mean 0.061 [0.047, 0.081] (seeds: 0.045, 0.055, 0.060, 0.100, 0.045)
- graph vs B-MCTS: mean 0.107 [0.087, 0.130] (seeds: 0.075, 0.110, 0.105, 0.150, 0.095)

Arm contrast — same-examples

Graph – grid difference of seed-level means; bootstrap 95% over seeds (3 per arm, independent).

- vs B-RND: graph–grid = -0.169 [-0.272, -0.062]

- vs B-HEU: graph-grid = -0.064 [-0.111, -0.017]
- vs B-MCTS: graph-grid = -0.009 [-0.055, +0.028]

Arm contrast — same-wallclock

Graph – grid difference of seed-level means; bootstrap 95% over seeds (3 per arm, independent).

- vs B-RND: graph-grid = -0.161 [-0.278, -0.048]
- vs B-HEU: graph-grid = -0.059 [-0.087, -0.029]
- vs B-MCTS: graph-grid = -0.018 [-0.066, +0.025]

H7 ablation results (feeds booklet table H-T3)

Reference: H6 graph full method (journal H6-2026-09-19-comparison-01).

Ablation	Component removed	Outcome
A1 naive adjacency (graph- untyped)	direction-typed edge matrices → one shared matrix	Training diverged (NaN, generation 0, 3/3 seeds) — untrainable at parity settings; eval tables are artifacts of a NaN policy and are excluded as scores (journal H7-2026-09-23-a1- divergence-01). Attribution: typed edges contribute at minimum optimization stability.
A2 no global pooling (graph- nogpool)	global-pooling bias in all layers	NULL effect: diff vs full graph -0.001 [-0.170, +0.165] (B-RND), -0.007 [-0.043, +0.030] (B- HEU), -0.013 [-0.043, +0.013] (B-MCTS); seed variance dominates; failure modes unchanged (journal H7-2026-10- 02-a2-nogpool-01). Pooling is dispensable at this scale.

A1' supplement (graph-untyped-clip, two-component, D-030) | untyped edges + grad-clip 1.0 | **Trains finite; scores within the full arm's band** — diffs vs full graph: B-RND -0.066 [-0.254, +0.130], B-HEU +0.032 [-0.011, +0.078], B-MCTS +0.008 [-0.040, +0.063]. NEVER attributed to typing alone (clip confound). With A1: typed edges' measurable contribution at this scale is concentrated in optimization stability (journal H7-2026-10-09-a1prime-01). |

Score / cost table (H6 task 7; fig2)

Metric	Grid arm	Graph arm
Parameters	1.44 M	1.47 M (+2.1%)
Best-provider inference (b1)	2.62 ms (CoreML)	3.67 ms (CPU)

Metric	Grid arm	Graph arm
Mean run wall-clock (10 gens × 500 games)	30.0 h	61.2 h (2.0×)
Mean self-play cost	21.6 s/game	44.0 s/game
Training throughput (MPS)	≈770 pos/s	≈195 pos/s
Population score, same-examples	0.411	0.331
Population score, same-wall-clock ($T^*=18.77$ h)	0.405	0.326

Population score = mean over the three frozen opponents of the seed-mean score (truncations excluded, reported separately in the results tables). Sources: results-*.csv, wallclock.json per run, comparison-controls.md measurements; journal H6-2026-09-19-comparison-01.

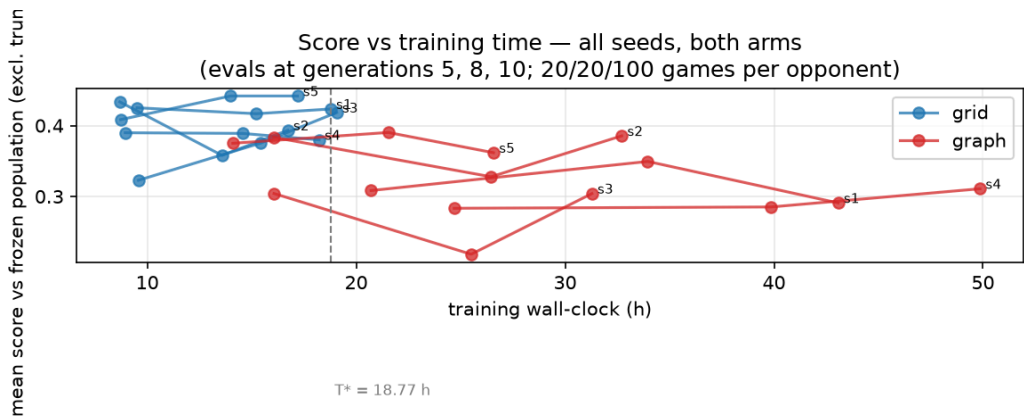


fig1-score-vs-time.png

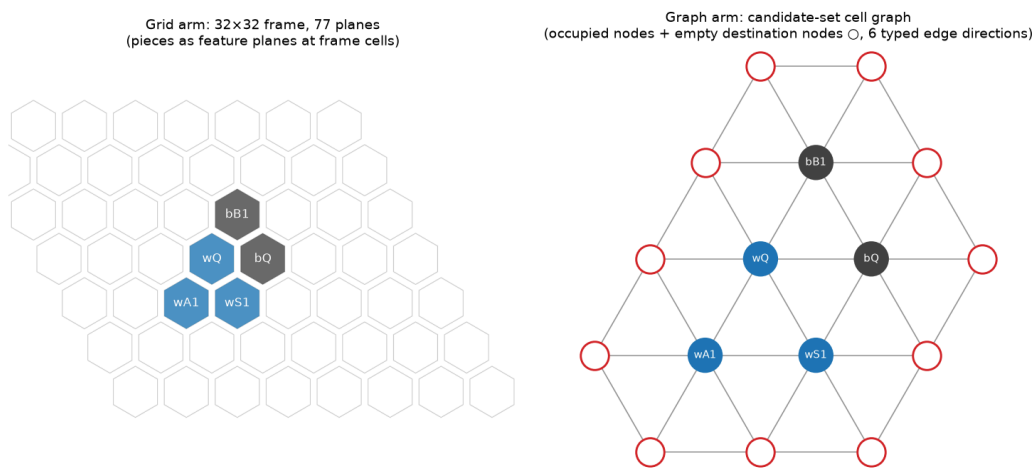


fig3-encodings.png

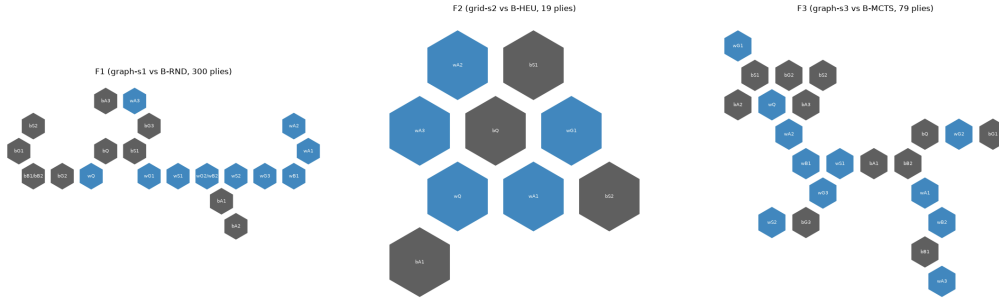


fig4-failures.png

Three commented failure positions (H6 task 7)

Selection criteria are mechanical, stated in `scripts/make_figures.py::fig4`, and applied over the raw per-game CSV records; each selected game is reproduced deterministically (fresh per-game engine seeds) and verified against its CSV row.

F1: graph vs B-RND — longest truncated game (a won position it cannot close: wins material, then shuffles to the 300-ply cap)

- game: graph-s1 vs B-RND, opening 2, A white, 300 plies
- reproduction verified against CSV row: YES
- GameString: Base;InProgress;White[151];wG1;bS1 \wG1;wQ /wG1;bQ -bS1;wS1 wG1-;bG1 -bQ;wG2 wS1-;bB1 bQ/;wS2 wG2-;bA1 -bG1;wG3 wS2-;bS2 \bA1;wA1 wG3\;bA2 bB1-;wB1 wA1\;bB1 bS1;wB2 wB1-;bB1 wG1;wA2 wB2\;bA2 bS2-;wA3 wB2/;bG2 \bS2;wA2 wA3/;bB2 bG2/;wA2 -bG2;bA1 /bQ;wA3 -wA2;bG3 bS1/;wA3 wB2\;bA3 \bG3;wA3 wB2/;bA1 /wS1;wA3 \bA3;bA1 wA3-;wB2 wA1-;bA1 wS2/;wB1 /wA1;bA1 bB2-;wA2 wA3/;bA1 \bB2;wA2 -bA1;bB1 bS1;wA2 wG3/;bA1 bB2\;wB1 wA1;bA1 /bG1;wB2 wB1;bA1 wG2/;wB2 wB1\;bA1 bG3-;wB2 /wB1;bA1 bG3\;wB2 -wB1;bB2 \bG2;wB...

F2: grid vs B-HEU — shortest decided loss (the heuristic's queen-targeting tactics strike before the net consolidates)

- game: grid-s2 vs B-HEU, opening 2, A black, 19 plies
- reproduction verified against CSV row: YES
- GameString: Base;WhiteWins;Black[10];wG1;bS1 \wG1;wQ /wG1;bQ -bS1;wA1 wG1-;bA1 bS1/;wA1 \bA1;bQ -wG1;wQ /bQ;bS2 -bS1;wA2 \wA1;bS2 /wQ;wA2 -bS1;bS2 wG1\;wA1 -bS2;bA1 /wA1;wA3 \wA2;bA1 /wQ;wA3 -bQ

F3: graph vs B-MCTS — longest drawn game (avoids losing without ever generating winning threats)

- game: graph-s3 vs B-MCTS, opening 19, A white, 79 plies
- reproduction verified against CSV row: YES
- GameString: Base;Draw;Black[40];wS1;bA1 wS1-;wA1 \wS1;bB1 bA1-;wA2 -wA1;bA2 /bB1;wQ \wA2;bQ bB1-;wQ wA2/;bA2 /wA2;wQ \wA2;bA2 -wQ;wB1 -wS1;bA3 bA1/;wA1 bQ-;bA3 bA2/;wA1 /bA1;bG1 bA1/;wA1 bQ-;bG2 -bA3;wA1 -bG2;bG3 /bB1;wA1 /bQ;bB1 wA1;wB2 /wS1;bG2 bA3-;wB2 wS1;bB2 -bQ;wB2 /wS1;bA3 bG2\;wB2 -bG3;bQ bB1-;wB2 /bG3;bS1 bG2-;wB2 /bB1;bS1 -bG2;wB2 /bQ;bQ bB2-;wB2 /bB1;bS2 bG2-;wB2 bB1\;bB1

```
wB2;wG1 /wS1;bQ bG1-;wG1 \bS1;bS2 wG1;/wA3 -wG1;bS2 -wA3;wG2 /wS1;bG2 -wA2;wG2
-bG1;bG2 bS1-;wG2 bQ-;bG1 wG2-;wG3 /wS1;bG3 wS1...
```

Annexes (pointers)

Bulky artifacts live in the repository; each annex names its identifiers and how to read them (plan ch. 23).

- **A. Position corpora.** `tests/critical_positions/` (30 rules-derived cases + README with sources and review status); `tests/tactical_positions/` (5 cases + runner).
- **B. Rule and encoding conventions.** `docs/representations/{grid,graph,comparison-controls}.md`; `docs/action-decoder.md`.
- **C. Architectures and hyperparameters.** `python/hivenet/model.py`, `python/hivenet/graph_model.py`; training flags in each run's `train-config.json`.
- **D. Configs, seeds, manifests.** `configs/` (matrix, baselines with sha256-pinned weights, pinned eval settings, ablation diffs); per-run `*-manifest.json` and `wallclock.json` under `data/runs/`.
- **E. Reproduction.** `scripts/reproduce_minimal.sh` (fresh-environment minimal scenario); `scripts/make_results.py` and `make_figures.py` regenerate every table and figure from raw per-game records.
- **F. Claims register.** `paper/claims.md` — one row per claim: claim, evidence, section, limit. No row, no claim.
- **G. Experiment journal.** `journal/` — every run and measurement, negative results included; decision log in `state/decisions.md` (workspace repository).

v1.0-draft — assembled 2026-10-09 by `scripts/assemble_booklet.py`; sources in `paper/` govern.